

HANDOUT PEMBELAJARAN — SERI 1 dari 3

Spring MVC

Aplikasi Web dengan Tampilan HTML

@Controller • Thymeleaf • Form Handling • Static Resources

Tujuan

Membuat web dinamis dengan tampilan HTML menggunakan Spring MVC

Durasi

1,5 Jam (90 Menit)

Tools

IntelliJ • Thymeleaf • Browser

Setelah belajar materi ini, mahasiswa mampu:

1. Memahami alur kerja Spring MVC dan perbedaannya dengan aplikasi biasa
2. Membuat project Spring Boot dengan dependensi Web dan Thymeleaf
3. Membuat @Controller yang mengirim data ke halaman HTML
4. Menggunakan Model, ModelAndView, dan RedirectAttributes
5. Membuat form HTML dan memproses input dari pengguna (Form Handling)
6. Menerapkan validasi input form di sisi server
7. Menyajikan file statis: CSS, JavaScript, dan gambar

Agenda Pembelajaran

Waktu	Durasi	Topik	Metode
0:00 – 0:05	5 menit	Pembukaan & overview Spring MVC	Presentasi
0:05 – 0:15	10 menit	Konsep MVC & cara kerja Spring MVC	Presentasi + Diskusi
0:15 – 0:25	10 menit	Setup project + struktur folder	Demo Live
0:25 – 0:40	15 menit	@Controller, Model & Thymeleaf dasar	Demo + Hands-on
0:40 – 0:55	15 menit	Form Handling & Validasi	Demo + Hands-on
0:55 – 1:05	10 menit	ModelAndView & RedirectAttributes	Demo + Hands-on
1:05 – 1:20	15 menit	Static resources (CSS/JS/gambar)	Demo + Hands-on
1:20 – 1:30	10 menit	Tanya jawab & ringkasan	Diskusi

1

Konsep Spring MVC

10 menit

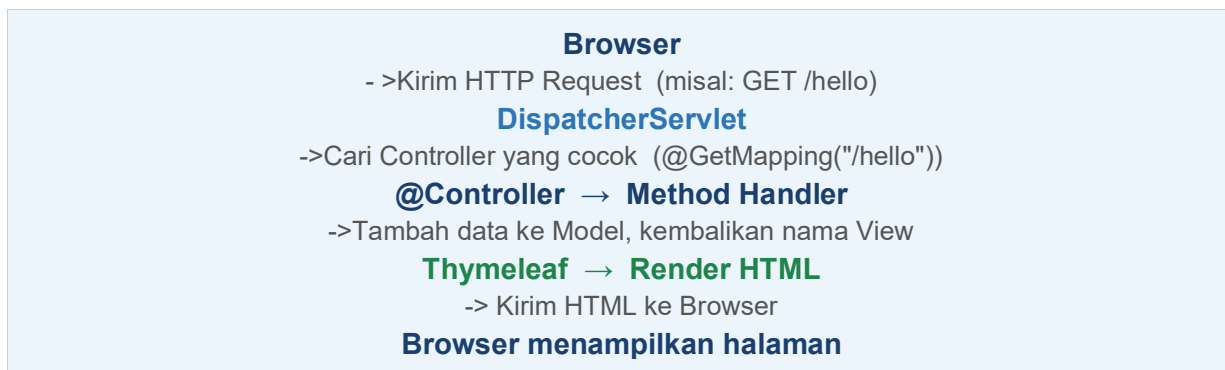
1.1 Apa itu MVC?

MVC adalah pola desain (design pattern) yang memisahkan aplikasi menjadi 3 bagian agar kode lebih rapi, terstruktur, dan mudah dipelihara.

Komponen	Singkatan	Tanggung Jawab	Contoh di Spring
Model	M	Menyimpan & membawa data	Class Java, List, Map
View	V	Menampilkan data ke pengguna (tampilan)	File HTML + Thymeleaf
Controller	C	Menerima request & mengatur alur	@Controller class

1.2 Bagaimana Spring MVC Bekerja?

Setiap request dari browser melewati alur berikut secara otomatis:



1.3 Perbedaan @Controller vs @RestController

Anotasi	Return	Digunakan untuk	Contoh return
@Controller	Nama file HTML (View)	Aplikasi web dengan tampilan HTML	return "index";
@RestController	Data langsung (JSON/teks)	REST API tanpa tampilan HTML	return user; → JSON

💡 Catatan

Materi ini menggunakan @Controller karena kita membuat halaman web yang terlihat di browser. Handout berikutnya akan membahas @RestController untuk REST API.

2.1 Buat Project di Spring Initializr

Buka <https://start.spring.io>, lalu isi pengaturan berikut:

Pengaturan	Nilai yang dipilih
Project	Maven
Language	Java
Spring Boot	3.x.x atau (versi terbaru)
Group	com.example
Artifact	webdemo
Packaging	Jar
Java	17

2.2 Dependensi yang Harus Ditambahkan

Nama Dependensi	Fungsi
Spring Web	Mengaktifkan Spring MVC & embedded Tomcat
Thymeleaf	Template engine untuk render HTML dinamis
Spring Boot DevTools	Hot reload — perubahan langsung terlihat tanpa restart
Lombok (opsional)	Mengurangi kode getter/setter yang repetitif

2.3 Struktur Folder Project

```
webdemo/  
├── src/  
│   ├── main/  
│   │   ├── java/com/example/webdemo/  
│   │   │   ├── WebdemoApplication.java ← Entry point  
│   │   │   └── controller/  
│   │   │       └── HomeController.java ← @Controller  
│   │   └── resources/  
│   │       ├── templates/ ← File HTML (Thymeleaf)  
│   │       │   ├── index.html  
│   │       │   └── about.html  
│   │       ├── static/ ← File statis  
│   │       │   ├── css/  
│   │       │   │   └── style.css  
│   │       │   ├── js/  
│   │       │   │   └── script.js  
│   │       │   └── images/  
│   │       └── application.properties  
└── pom.xml
```

 Penting — Lokasi File HTML

File HTML Thymeleaf HARUS disimpan di folder: `src/main/resources/templates/` File statis (CSS, JS, gambar) HARUS di: `src/main/resources/static/` Jika salah folder, Spring Boot tidak akan menemukan file tersebut!

3.1 Membuat Controller Pertama

Controller adalah class Java biasa yang diberi anotasi `@Controller`. Setiap method di dalamnya menangani satu URL tertentu.

```
package com.example.webdemo.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

@Controller // Tandai class ini sebagai Spring MVC Controller
public class HomeController {

    // Tangani request: GET /
    // Ketika user buka http://localhost:8080/
    @GetMapping("/")
    public String index() {
        return "index"; // ← Cari file: templates/index.html
    }

    // Tangani request: GET /hello
    @GetMapping("/hello")
    public String hello(Model model) {
        // Kirim data ke halaman HTML
        model.addAttribute("nama", "Budi Santoso");
        model.addAttribute("pesan", "Selamat datang di Spring MVC!");
        return "hello"; // ← Cari file: templates/hello.html
    }
}
```

3.2 Template HTML dengan Thymeleaf

Thymeleaf adalah template engine yang memungkinkan kita menyisipkan data Java ke dalam HTML menggunakan atribut `th:*`.

File: templates/hello.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Halaman Hello</title>
</head>
<body>

    <!-- th:text → tampilkan nilai dari Model -->
    <h1 th:text="${pesan}">Pesan default jika tidak ada data</h1>

    <!-- Menggabungkan teks dengan nilai Model -->
    <p>Halo, <strong th:text="${nama}">Nama</strong>! Selamat belajar.</p>
```

```

<!-- th:if → tampilkan elemen hanya jika kondisi true -->
<p th:if="{nama != null}">Data nama berhasil dikirim dari Controller.</p>

<!-- Link ke halaman lain -->
<a href="/">Kembali ke Beranda</a>

</body>
</html>

```

3.3 Sintaks Thymeleaf yang Wajib Diketahui

Sintaks	Fungsi	Contoh
<code>{variabel}</code>	Tampilkan nilai variabel dari Model	<code>th:text="{nama}"</code>
<code>th:text</code>	Set teks konten elemen HTML	<code><p th:text="{pesan}"></code>
<code>th:if</code>	Tampilkan elemen jika kondisi true	<code>th:if="{user != null}"</code>
<code>th:unless</code>	Tampilkan elemen jika kondisi false	<code>th:unless="{loggedIn}"</code>
<code>th:each</code>	Iterasi / loop data list	<code>th:each="item : {items}"</code>
<code>th:href</code>	Set nilai atribut href (link)	<code>th:href="@{/about}"</code>
<code>th:src</code>	Set nilai atribut src (gambar)	<code>th:src="@{/images/logo.png}"</code>
<code>th:value</code>	Set nilai atribut value (form)	<code>th:value="{user.name}"</code>
<code>th:action</code>	Set URL tujuan form	<code>th:action="@{/simpan}"</code>
<code>th:object</code>	Bind object ke form	<code>th:object="{user}"</code>

3.4 Menampilkan List Data dengan th:each

```

// Controller — kirim list ke HTML
@GetMapping("/mahasiswa")
public String daftarMahasiswa(Model model) {
    List<String> namaMahasiswa = List.of(
        "Budi Santoso",
        "Siti Rahayu",
        "Ahmad Fauzi"
    );
    model.addAttribute("daftarNama", namaMahasiswa);
    return "mahasiswa";
}

```

File: templates/mahasiswa.html

```

<ul>
    <!-- th:each = loop, seperti for-each di Java -->
    <li th:each="nama : {daftarNama}"
        th:text="{nama}">Nama mahasiswa</li>
</ul>

<!-- Hasil render di browser: -->

```

```
<!-- <li>Budi Santoso</li>      -->  
<!-- <li>Siti Rahayu</li>       -->  
<!-- <li>Ahmad Fauzi</li>      -->
```


4.1 Model vs ModelAndView

Ada dua cara mengirim data dari Controller ke View:

Cara	Penjelasan	Kapan Digunakan
Model (parameter)	Data dan nama view dipisah. Method return String (nama view).	Paling umum, kode lebih ringkas
ModelAndView (return)	Data dan nama view digabung dalam satu objek return.	Ketika nama view ditentukan secara kondisional

4.2 Contoh ModelAndView

```
import org.springframework.web.servlet.ModelAndView;

@GetMapping("/profil")
public ModelAndView profil() {
    ModelAndView mav = new ModelAndView();

    // Tentukan View mana yang akan ditampilkan
    mav.setViewName("profil"); // → templates/profil.html

    // Tambahkan data yang ingin dikirim ke View
    mav.addObject("judul", "Halaman Profil");
    mav.addObject("nip", "12345678");
    mav.addObject("divisi", "Teknologi Informasi");

    return mav;
}
```

4.3 RedirectAttributes — Kirim Pesan Setelah Redirect

Ketika form berhasil diproses, biasanya kita redirect ke halaman lain. RedirectAttributes digunakan untuk mengirim pesan singkat (flash message) ke halaman tujuan redirect — pesan ini otomatis hilang setelah dibaca sekali.

```
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

@PostMapping("/simpan")
public String simpanData(@ModelAttribute User user,
                        RedirectAttributes redirectAttrs) {
    // ... proses simpan data ...

    // addFlashAttribute: data tersedia sekali setelah redirect
    redirectAttrs.addFlashAttribute("sukses", "Data berhasil disimpan!");

    // Redirect ke halaman daftar
    return "redirect:/daftar";
}
```

```
// Halaman tujuan redirect
@GetMapping("/daftar")
public String daftar(Model model) {
    // Pesan flash otomatis tersedia di model
    return "daftar";
}
```

Di HTML — Tampilkan flash message:

```
<!-- Tampilkan pesan sukses jika ada -->
<div th:if="{sukses}"
      th:text="{sukses}"
      style="color: green; border: 1px solid green; padding: 10px;">
    Pesan sukses muncul di sini
</div>
```

Method	Kapan Data Tersedia	Hilang Setelah Dibaca?
addFlashAttribute(key, val)	Hanya di request berikutnya (setelah redirect)	Ya — otomatis hilang
addAttribute(key, val)	Ditambahkan ke URL sebagai query param	Tidak — tetap di URL
model.addAttribute(key, val)	Hanya di request saat itu (tanpa redirect)	Ya — selesai request

5.1 Alur Form Handling di Spring MVC

Membuat form di Spring MVC melibatkan dua langkah: menampilkan form (GET) dan memproses form (POST).

Step 1 — Tampilkan Form (GET)

8. User buka /form
9. Controller buat objek kosong
10. Tambahkan ke Model
11. Tampilkan halaman form.html

Step 2 — Proses Form (POST)

12. User isi form & klik Submit
13. Data dikirim ke /simpan (POST)
14. Controller terima & validasi
15. Redirect ke halaman lain

5.2 Model Class (POJO)

```
package com.example.webdemo.model;

import jakarta.validation.constraints.*;
import lombok.*;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class Mahasiswa {

    // @NotBlank → tidak boleh kosong atau hanya spasi
    @NotBlank(message = "Nama tidak boleh kosong")
    @Size(min = 3, max = 50, message = "Nama harus 3-50 karakter")
    private String nama;

    // @Email → format harus berupa email yang valid
    @Email(message = "Format email tidak valid")
    @NotBlank(message = "Email tidak boleh kosong")
    private String email;

    // @Min / @Max → batas nilai angka
    @Min(value = 17, message = "Usia minimal 17 tahun")
    @Max(value = 99, message = "Usia maksimal 99 tahun")
    private Integer usia;

    // @NotNull → tidak boleh null
    @NotNull(message = "Jurusan harus dipilih")
    private String jurusan;
}
```

5.3 Controller dengan Form Handling

```
import org.springframework.validation.BindingResult;
import jakarta.validation.Valid;

@Controller
```

```
@RequestMapping("/mahasiswa")
public class MahasiswaController {

    // GET /mahasiswa/form → tampilkan form kosong
    @GetMapping("/form")
    public String tampilkanForm(Model model) {
        // Buat objek kosong untuk di-bind ke form
        model.addAttribute("mahasiswa", new Mahasiswa());
        return "form-mahasiswa";
    }

    // POST /mahasiswa/simpan → proses data form
    // @Valid → aktifkan validasi dari anotasi di class Mahasiswa
    // BindingResult → tampung hasil validasi (HARUS tepat setelah @Valid)
    @PostMapping("/simpan")
    public String simpan(@Valid @ModelAttribute("mahasiswa") Mahasiswa mhs,
                        BindingResult result,
                        RedirectAttributes redirectAttrs) {

        // Cek apakah ada error validasi
        if (result.hasErrors()) {
            // Kembalikan ke form, error otomatis ditampilkan di HTML
            return "form-mahasiswa";
        }

        // TODO: simpan data ke database
        System.out.println("Data tersimpan: " + mhs.getNama());

        redirectAttrs.addFlashAttribute("sukses",
            "Mahasiswa " + mhs.getNama() + " berhasil didaftarkan!");
        return "redirect:/mahasiswa/daftar";
    }

    // GET /mahasiswa/daftar → halaman daftar
    @GetMapping("/daftar")
    public String daftar() {
        return "daftar-mahasiswa";
    }
}
```

5.4 Form HTML dengan Thymeleaf

File: templates/form-mahasiswa.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Form Mahasiswa</title>
    <link rel="stylesheet" th:href="@{/css/style.css}">
</head>
<body>
    <h1>Form Pendaftaran Mahasiswa</h1>

    <!-- th:action = URL tujuan POST, th:object = bind ke object Mahasiswa -->
    <form th:action="@{/mahasiswa/simpan}"
        th:object="${mahasiswa}"
```

```

        method="post">

<!-- Kolom Nama -->
<div>
    <label>Nama Lengkap:</label>
    <!-- th:field = bind ke field nama di object Mahasiswa -->
    <input type="text" th:field="*{nama}" placeholder="Masukkan nama">
    <!-- th:errors = tampilkan pesan error validasi jika ada -->
    <span th:if="{#fields.hasErrors('nama')}}"
          th:errors="*{nama}"
          style="color: red;">Error nama</span>
</div>

<!-- Kolom Email -->
<div>
    <label>Email:</label>
    <input type="email" th:field="*{email}" placeholder="Masukkan email">
    <span th:if="{#fields.hasErrors('email')}}"
          th:errors="*{email}" style="color: red;">Error email</span>
</div>

<!-- Kolom Usia -->
<div>
    <label>Usia:</label>
    <input type="number" th:field="*{usia}">
    <span th:if="{#fields.hasErrors('usia')}}"
          th:errors="*{usia}" style="color: red;">Error usia</span>
</div>

<!-- Dropdown Jurusan -->
<div>
    <label>Jurusan:</label>
    <select th:field="*{jurusan}">
        <option value="">-- Pilih Jurusan --</option>
        <option value="TI">Teknik Informatika</option>
        <option value="SI">Sistem Informasi</option>
        <option value="TK">Teknik Komputer</option>
    </select>
    <span th:if="{#fields.hasErrors('jurusan')}}"
          th:errors="*{jurusan}" style="color: red;">Error jurusan</span>
</div>

    <button type="submit">Simpan</button>
</form>

</body>
</html>

```

5.5 Anotasi Validasi — Referensi

Anotasi	Berlaku untuk	Fungsi
@NotNull	Semua tipe	Tidak boleh null
@NotBlank	String	Tidak boleh null, kosong, atau hanya spasi
@NotEmpty	String, Collection	Tidak boleh null atau kosong

@Size(min,max)	String, Collection	Panjang/ukuran antara min dan max
@Min(value)	Number	Nilai tidak boleh kurang dari value
@Max(value)	Number	Nilai tidak boleh melebihi value
@Email	String	Harus format email yang valid
@Pattern(regex)	String	Harus sesuai pola regex
@Positive	Number	Harus bernilai positif (> 0)
@Past / @Future	Date	Harus tanggal masa lalu / masa depan

Tambahkan Dependensi Validasi

Untuk menggunakan anotasi @NotBlank, @Email, dll, tambahkan ke pom.xml: <dependency>
<groupId>org.springframework.boot</groupId> <artifactId>spring-boot-starter-validation</artifactId> </dependency>

6.1 Apa itu Static Resources?

Static resources adalah file yang dikirim langsung ke browser tanpa diproses server — yaitu CSS (style), JavaScript (interaksi), dan gambar.

Lokasi Folder Static Resources

Semua file statis harus disimpan di: `src/main/resources/static/` Spring Boot otomatis menyajikan file di folder ini melalui URL:

- `src/main/resources/static/css/style.css` → `http://localhost:8080/css/style.css`
- `src/main/resources/static/js/script.js` → `http://localhost:8080/js/script.js`
- `src/main/resources/static/images/logo.png` → `http://localhost:8080/images/logo.png`

6.2 Contoh File CSS

File: `static/css/style.css`

```
/* Reset dasar */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: Arial, sans-serif;
  background-color: #f5f5f5;
  color: #333;
}

.container {
  max-width: 800px;
  margin: 30px auto;
  background: white;
  padding: 30px;
  border-radius: 8px;
  box-shadow: 0 2px 8px rgba(0,0,0,0.1);
}

h1 { color: #1A3F6F; margin-bottom: 20px; }

input, select {
  width: 100%;
  padding: 8px;
  margin: 6px 0 14px;
  border: 1px solid #ccc;
  border-radius: 4px;
}

button {
  background: #2E75B6;
  color: white;
  padding: 10px 24px;
```

```

    border: none;
    border-radius: 4px;
    cursor: pointer;
}

button:hover { background: #1A3F6F; }

.sukses {
    background: #d4edda;
    color: #155724;
    padding: 12px;
    border-radius: 4px;
    margin-bottom: 16px;
}

.error { color: red; font-size: 13px; }

```

6.3 Cara Memanggil di HTML Thymeleaf

```

<head>
    <!-- CSS — gunakan th:href dengan @{/...} -->
    <link rel="stylesheet" th:href="@{/css/style.css}">
</head>
<body>

    <!-- Gambar — gunakan th:src dengan @{/...} -->
    

    <!-- JavaScript — gunakan th:src dengan @{/...} -->
    <script th:src="@{/js/script.js}"></script>

    <!-- HINDARI path statis seperti ini (bisa error di sub-path): -->
    <!-- <link rel="stylesheet" href="/css/style.css"> ← kurang aman -->
    <!-- Selalu gunakan th:href="@{...}" agar Thymeleaf resolve path-nya -->

</body>

```

6.4 Contoh Halaman Lengkap dengan CSS

File: templates/index.html

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Beranda</title>
    <link rel="stylesheet" th:href="@{/css/style.css}">
</head>
<body>
    <div class="container">

        <!-- Flash message sukses -->
        <div class="sukses" th:if="${sukses}" th:text="${sukses}"></div>

        <h1>Sistem Pendaftaran Mahasiswa</h1>
    </div>

```



```
<p>Selamat datang di sistem kami.</p>

<a href="/mahasiswa/form">
  <button>Daftar Mahasiswa Baru</button>
</a>



</div>
<script th:src="@{/js/script.js}"></script>
</body>
</html>
```

Ringkasan & Checklist

✓	Kompetensi
☐	Memahami alur MVC: Request → DispatcherServlet → Controller → View
☐	Membuat project Spring Boot dengan dependensi Web dan Thymeleaf
☐	Membuat @Controller dan method handler dengan @GetMapping / @PostMapping
☐	Mengirim data dari Controller ke View menggunakan Model
☐	Menggunakan ModelAndView untuk menggabungkan data dan nama view
☐	Menggunakan RedirectAttributes untuk flash message setelah redirect
☐	Membuat form HTML dengan Thymeleaf (th:object, th:field, th:errors)
☐	Menerapkan validasi input dengan @Valid, @NotBlank, @Email, dll
☐	Menyimpan dan memanggil file statis (CSS, JS, gambar) dengan benar

Latihan Mandiri

Buat aplikasi web sederhana:

- Form registrasi dengan field Nama, Email, Password, dan Tanggal Lahir.
- Validasi: semua field wajib diisi, email harus valid, password minimal 8 karakter.
- Setelah submit sukses, redirect ke halaman 'Terima Kasih' dengan flash message.
- Tambahkan CSS agar tampilan rapi.

Referensi	URL
Dokumentasi Thymeleaf	https://www.thymeleaf.org/documentation.html
Spring MVC Guide	https://spring.io/guides/gs/serving-web-content
Validasi Spring	https://spring.io/guides/gs/validating-form-input